Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/Sham-2005/MLA0403-Deep_learning

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA04**
SLOT :

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)
CO Weightage: 25%

Duration: 90 minutes
Marks: 25

Assessment Tool Weightage: 40% ***Code of Conduct:***
I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

Criteria	Selection of Studies/Parameters (2)	Comparison & Differentiation (2.5)	Critical Evaluation (2.5)	Synthesis & Insight Generation (2)	Clarity & Presentation *(1)*	Total (10)
Weightage	*20 %*	*25%*	*25%*	*20 %*	*10 %*	100 %
Q1						
Q2						
Q3						
Q4						

Q5						
Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS: 05 **Total Marks:25**

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q1. CNN vs Fully Connected Neural Network

For classifying **X-ray images**, a CNN is generally much more suitable than a traditional fully connected neural network because CNNs are specifically designed to exploit the **spatial structure of images**.

1. Architectural Design

Fully Connected Neural Network (FCNN)

In a fully connected network, every neuron in one layer is connected to every neuron in the next layer.

For an image:

```
X-ray Image
     ↓
Flatten Image
     ↓
Fully Connected Layer
     ↓
Fully Connected Layer
     ↓
Output
     ↓
Normal / Disease
```

The 2D image must first be converted into a **1D vector**.

CNN

A CNN preserves the spatial structure of the image.

```
X-ray Image
     ↓
Convolution
     ↓
ReLU
     ↓
Pooling
     ↓
Convolution
     ↓
ReLU
     ↓
Pooling
     ↓
Fully Connected / Classification Head
     ↓
Normal / Disease
```

---

2. Motivation

The main motivation for using CNNs is that images contain **local spatial relationships**.

For example, in an X-ray:

- Nearby pixels are related.
- Edges form anatomical structures.
- Textures can indicate abnormalities.
- Different regions combine to form meaningful patterns.

A fully connected network does not explicitly exploit these relationships. CNNs are designed to do so automatically.

---

3. Spatial Feature Extraction

CNNs automatically learn hierarchical features.

```
Early Layers
     ↓
Edges and Corners
     ↓
Middle Layers
     ↓
Textures and Shapes
     ↓
Deep Layers
     ↓
Complex Anatomical Patterns
     ↓
Disease Classification
```

For X-rays, early layers may detect edges and boundaries, while deeper layers can learn more complex structures associated with disease patterns.

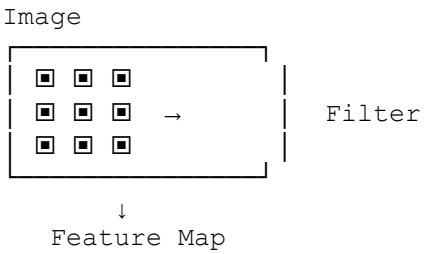
A traditional FCNN has no built-in mechanism specifically designed for this type of spatial feature extraction.

---

4. Role of Convolutional Layers

A convolutional layer applies small filters to different regions of the image.

For example, a **3 × 3 filter** slides across the image:



The convolution operation produces a **feature map** showing where the learned feature occurs.

Different filters can learn different patterns such as:

- Horizontal edges
- Vertical edges
- Curves
- Textures
- Shapes

The filter weights are learned automatically during training.

---

5. Role of Filters

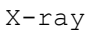
Filters are small learnable matrices.

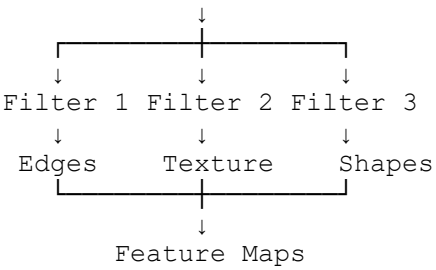
For example:

[
3\times3 = 9
]

weights per input channel, plus a bias.

Different filters detect different visual characteristics.





As the network becomes deeper, the filters learn increasingly complex features.

---

6. Role of Pooling Layers

Pooling reduces the spatial dimensions of feature maps.

For example, **2 × 2 max pooling** selects the largest value from each region.

```
2 × 2 region

[2  5]
[1  3]

     ↓

     5
```

Benefits

- Reduces feature-map size
- Reduces computational requirements
- Reduces the number of parameters in later layers
- Retains important features
- Provides some robustness to small translations

---

7. Number of Parameters

This is one of the biggest differences between CNNs and FCNNs.

Suppose an X-ray has:

[
224\times224 = 50,176
]

pixels.

If it is connected to a fully connected layer containing 1,000 neurons:

$$
50,176\times1,000
=50,176,000
$$

weights are required, excluding biases.

CNN

Suppose a CNN uses a **3 × 3 filter**.

Only:

$$
3\times3=9
$$

weights are required per filter per input channel.

The same filter is reused across the entire image. This is called **parameter sharing**.

Therefore, CNNs generally require **far fewer parameters** than an equivalent fully connected network.

---

8. Computational Complexity

Fully Connected Network

A large number of connections results in:

- High memory requirements
- High computational cost
- More trainable parameters
- Greater risk of overfitting

CNN

CNNs use:

- Local connectivity
- Small filters
- Parameter sharing
- Pooling/downsampling

Therefore, they are much more computationally efficient for image data.

---

9. CNN vs Fully Connected Network

Feature	Fully Connected NN	CNN
Input handling	Flattened vector	2D/3D image structure preserved
Spatial information	Not explicitly preserved	Preserved
Feature extraction	Usually learned through dense connections	Automatically learned using filters
Local relationships	Poorly exploited	Strongly exploited
Parameters	Very high	Much lower
Parameter sharing	No	Yes
Computational cost	High for large images	Lower
Pooling	Usually absent	Commonly used
Translation robustness	Limited	Better
Image classification	Less suitable	Highly suitable

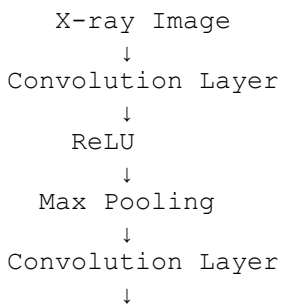
---

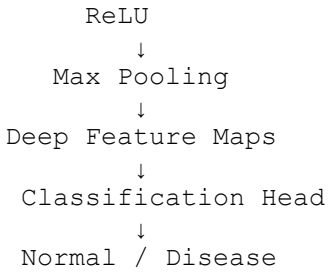
10. Why CNNs Are More Suitable for X-ray Classification

CNNs are preferred because they can:

1. **Preserve spatial relationships** between pixels.
2. Automatically learn useful visual features.
3. Detect local patterns using convolutional filters.
4. Build hierarchical representations through multiple layers.
5. Reduce parameters through parameter sharing.
6. Reduce computation using pooling/downsampling.
7. Handle high-dimensional image inputs efficiently.
8. Generalize better to objects/features appearing in different image locations.
9. Learn complex patterns without manually designing features.
10. Provide strong performance for medical image classification when appropriately trained and validated.

Overall Architecture





Conclusion

A **CNN is more suitable than a traditional fully connected neural network for X-ray image classification** because it preserves spatial information, automatically learns hierarchical visual features, uses parameter sharing to dramatically reduce the number of trainable parameters, and uses pooling to reduce computational complexity. Therefore, CNNs provide a more efficient and effective architecture for extracting meaningful patterns from medical images.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q2. Different CNN Layers and Filters

A CNN-based defect detection system can automatically learn visual patterns from product images and classify them as **normal or defective**. Each CNN layer has a different role, and together they create a hierarchical feature-learning process.

1. Functions of Different CNN Layers

Layer	Main Function	Spatial Information	Computational Requirement	Role in Defect Detection
Convolutional	Extract features using filters	Preserves spatial relationships	Moderate	Detects edges, textures, cracks, scratches
Activation (ReLU)	Introduces non-linearity	Maintains feature-map dimensions	Low	Helps learn complex defect patterns
Pooling	Reduces spatial dimensions	Partially reduces spatial information	Low	Reduces computation while

Layer	Main Function	Spatial Information	Computational Requirement	Role in Defect Detection
				retaining important features
Fully Connected	Combines learned features for classification	Spatial structure mostly removed	High	Produces final defect classification

---

2. Convolutional Layers

The convolutional layer is the main **feature extraction layer**.

It applies small learnable filters to different regions of the product image.

```
Product Image
      ↓
Convolution Filters
      ↓
Feature Maps
      ↓
Edges / Textures / Patterns
```

For example, filters can learn to detect:

- Edges
- Lines
- Corners
- Cracks
- Scratches
- Surface irregularities

Spatial Information

Convolution preserves the spatial arrangement of features. Therefore, the network can understand **where a defect occurs** in the image.

Computational Requirement

CNNs use local connections and parameter sharing, so they require far fewer parameters than a fully connected network applied directly to the entire image.

---

3. Activation Layers

After convolution, an activation function such as **ReLU** is normally applied.

$$
\mathrm{ReLU}(x)=\max(0,x)
$$

It introduces **non-linearity** into the network.

Without activation functions, multiple convolutional layers would effectively behave like a linear transformation and would have limited ability to learn complex patterns.

Role in defect detection

ReLU allows the network to learn complicated relationships such as:

```
Edge + Texture + Shape
          ↓
Possible Crack Pattern
```

Activation layers are computationally inexpensive and normally preserve the dimensions of the feature map.

---

4. Pooling Layers

Pooling reduces the spatial dimensions of feature maps.

A common method is **Max Pooling**.

For a 2 × 2 region:

```
[2  5]
[1  3]
   ↓
   5
```

Functions

- Reduces feature-map dimensions
- Reduces computational requirements
- Reduces memory usage
- Retains strong/important features
- Provides some robustness to small changes in defect position

For example:

```
224 × 224
    ↓ Pooling
112 × 112
    ↓ Pooling
 56 × 56
```

However, excessive pooling can remove useful fine-grained defect information, so it must be used carefully when detecting very small cracks or scratches.

---

5. Fully Connected Layers

After several convolution and pooling stages, the learned feature maps are converted into a compact representation using **Flattening or Global Average Pooling**.

The features are then passed to a classification layer.

```
Feature Maps
     ↓
Flatten / Global Average Pooling
     ↓
Fully Connected Layer
     ↓
Output Layer
     ↓
Normal / Defective
```

The fully connected layer combines the learned features and makes the final classification decision.

For binary classification, a **sigmoid** output can be used:

[
P(Defective)
]

---

6. Early-Layer Filters vs Deeper-Layer Filters

CNN filters learn different types of features depending on their depth.

Early-Layer Filters

The first few convolutional layers operate directly on the input image.

They typically learn **simple, low-level features** such as:

- Horizontal edges
- Vertical edges
- Diagonal edges
- Corners
- Simple textures

- Basic color/intensity transitions

Example:

```
Product Image
     ↓
Early Filters
     ↓
Edges + Lines + Basic Textures
```

These features are general and can be useful for many types of images.

Middle-Layer Filters

Middle layers combine low-level features to learn more meaningful structures.

For an industrial product, they may detect:

- Repeated surface patterns
- Shapes
- Curves
- Texture combinations
- Small crack structures
- Scratch patterns

```
Edges + Textures
       ↓
Middle Layers
       ↓
Defect Components
```

Deeper-Layer Filters

Deeper layers combine the features learned by earlier layers into **high-level representations**.

They may recognize patterns such as:

- Large cracks
- Complex surface damage
- Broken components
- Deformed product regions
- Specific defect types

```
Edges
  ↓
Textures
  ↓
Shapes
  ↓
Complex Defect Pattern
```

```
  ↓
Defect Classification
```

---

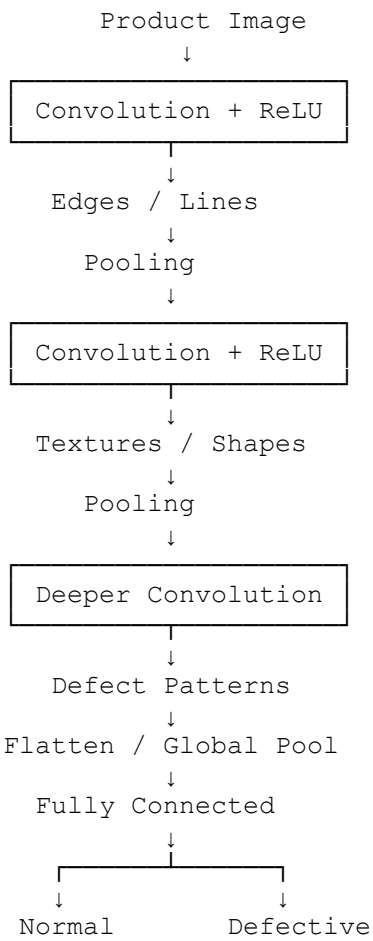
7. Comparison of Filter Learning

Feature	Early Filters	Deeper Filters
Feature level	Low-level	High-level
Typical features	Edges, corners	Defect structures
Complexity	Simple	Complex
Reusability	Highly general	More task-specific
Spatial understanding	Local	More global
Example	Scratch edge	Complete scratch/defect pattern

---

8. How All Layers Work Together

The layers form a **hierarchical representation-learning system**.



Why this improves defect detection

Convolution finds important visual patterns.

Activation enables the network to learn nonlinear and complex relationships.

Pooling reduces unnecessary spatial information and computation while retaining strong features.

Deeper filters combine simple features into complex defect representations.

Fully connected layers combine the learned representations to make the final classification.

Conclusion

A CNN detects industrial defects effectively because its layers work together in a **hierarchical feature-learning process**. Early filters identify simple features such as edges and textures, while deeper filters combine them into complex defect patterns. Convolution preserves important spatial information, pooling reduces computational requirements, activation functions introduce non-linearity, and fully connected layers perform the final classification. Thus, the complete CNN can automatically learn **normal and defective product patterns** without manually designing every defect feature.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for largescale image-processing applications.

Q3. Parameter Sharing vs Independent Weights

For a smart surveillance system processing thousands of high-resolution images, **parameter sharing in CNNs** is much more efficient than using independent weights in a fully connected neural network. The main reason is that CNNs reuse the same small filters across the entire image.

1. Parameter Sharing in CNNs

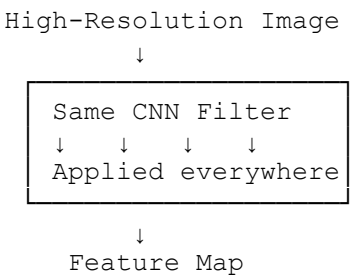
In a CNN, a convolutional filter has a small number of learnable weights and is applied repeatedly at different locations.

For example, a **3 × 3 filter** contains:

[
3\times3=9
]

weights per input channel, plus a bias.

The same filter is then moved across the entire image.



This is called **parameter sharing**.

---

2. Independent Weights in Fully Connected Networks

In a fully connected network, every input pixel has its own connection to each neuron.

Suppose an image is:

[
1920\times1080
]

The number of pixels is:

[
1920\times1080=2,073,600
]

If this image is connected to a fully connected layer containing 1,000 neurons:

[
2,073,600\times1,000

2.0736\ billion
]

weights would be required, excluding biases.

This creates a very large parameter and memory requirement.

---

3. Comparison of Parameters

Factor	CNN – Parameter Sharing	Fully Connected – Independent Weights
Weight sharing	Yes	No
Number of parameters	Much smaller	Extremely large
Memory requirement	Lower	Very high
Computational cost	Lower	Very high
Spatial structure	Preserved	Lost after flattening
Feature detection	Local and hierarchical	Less spatially specialized
Translation handling	Better	Poorer
Scalability	High	Difficult for high-resolution images

---

4. Memory Requirements

The number of parameters directly affects memory requirements.

Fully Connected Network

Millions or billions of independent weights may need to be stored.

This causes:

- High RAM/VRAM usage
- Large model size
- Higher data-transfer requirements
- Greater deployment cost

CNN

CNNs use a relatively small collection of filters and reuse them throughout the image.

For example, if a CNN has 64 filters of size 3 × 3 for a grayscale image:

[
64\times3\times3=576
]

weights are required, plus biases.

Thus, the memory requirement can be dramatically smaller than an equivalent fully connected approach.

---

5. Computational Complexity

Fully connected networks perform calculations for a huge number of connections.

For a high-resolution image:

```
Millions of Pixels
        ↓
Every Pixel Connected
        ↓
Thousands of Neurons
        ↓
Huge Number of Operations
```

This becomes expensive when thousands of surveillance images must be processed.

CNNs use local receptive fields:

```
Image
 ↓
Small Region
 ↓
Filter
 ↓
Feature
 ↓
Move Filter
 ↓
Next Region
```

Only local regions are processed at a time, and the same filter is reused.

Therefore, CNNs are significantly more computationally efficient for image processing.

---

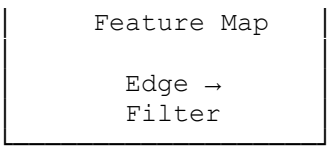
6. Feature Detection

Parameter sharing also makes CNNs very effective at detecting visual features.

Suppose a filter learns to detect an edge.

The same filter can detect the edge anywhere:

```
        Image
┌──────────────────────┐
│ Edge → Filter        │
│         ↓            │
```



The network does not need a separate edge detector for every possible location.

This allows CNNs to efficiently learn:

- Edges
- Corners
- Textures
- Faces
- Vehicles
- Objects
- Human activities

---

7. Translation-Related Invariance

An important advantage of parameter sharing is that a feature can be detected even when its position changes.

For example, a surveillance camera may see a car:

```
Image 1: Car → Left

Image 2: Car → Center

Image 3: Car → Right
```

Because the same convolutional filter scans across the entire image, it can recognize similar features at different locations.

This provides **translation-related robustness**.

Pooling can further increase this robustness by making the representation less sensitive to small shifts in object position.

Strictly speaking, convolution provides **translation equivariance**—the feature map shifts when the input shifts—while pooling and later processing can provide some degree of **translation invariance**.

---

8. Learning Efficiency

Parameter sharing means that a single filter learns from many different locations in every training image.

For example:

```
Image
 ├── Region 1 → Same Filter
 ├── Region 2 → Same Filter
 ├── Region 3 → Same Filter
 ├── Region 4 → Same Filter
 └── Region 5 → Same Filter
```

Therefore, the filter receives learning signals from many spatial locations.

This provides several advantages:

- Faster feature learning
- Fewer parameters to optimize
- Lower risk of overfitting
- Better generalization
- Efficient use of training data

---

9. Why It Is Important for Large-Scale Surveillance

A smart surveillance system may process **thousands or millions of high-resolution frames**.

Without parameter sharing:

```
High-Resolution Images
        ↓
Huge Number of Parameters
        ↓
High Memory Usage
        ↓
High Computation
        ↓
Slow Processing
```

With CNN parameter sharing:

```
High-Resolution Images
        ↓
Small Shared Filters
        ↓
Efficient Feature Extraction
        ↓
Lower Memory + Computation
        ↓
Fast Image Processing
```

This makes CNNs suitable for applications such as:

- CCTV surveillance

- Face detection
- Vehicle detection
- Pedestrian detection
- Intrusion detection
- Traffic monitoring

---

10. Overall Comparison

Aspect	CNN Parameter Sharing	Fully Connected Independent Weights
Parameters	Low	Very high
Memory	Low	Very high
Computation	Efficient	Expensive
Spatial information	Preserved	Lost after flattening
Feature reuse	Excellent	Poor
Feature detection	Location-aware	Less spatially efficient
Translation robustness	Good	Limited
Overfitting risk	Generally lower	Generally higher with huge parameter counts
High-resolution images	Suitable	Difficult
Large-scale processing	Highly suitable	Computationally expensive

Conclusion

Parameter sharing is one of the fundamental reasons CNNs are efficient for large-scale image processing. Instead of learning separate weights for every pixel-location connection, a CNN learns a small set of filters and reuses them throughout the image. This dramatically reduces the **number of parameters, memory usage, and computational cost**, while allowing the same visual features to be detected at different locations. For a smart surveillance system processing thousands of high-resolution images, this makes CNNs far more practical and scalable than fully connected networks.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

Q4. AlexNet vs ResNet

Both **AlexNet** and **ResNet** are important CNN architectures used for image recognition. However, they differ significantly in depth, architecture, training, parameter efficiency, and feature-learning capability.

1. Basic Architecture

AlexNet

AlexNet is an early deep CNN architecture that became highly successful for large-scale image classification.

A simplified structure is:

```
Input Image
     ↓
Convolution
     ↓
ReLU
     ↓
Pooling
     ↓
Convolution
     ↓
ReLU
     ↓
Pooling
     ↓
Convolution Layers
     ↓
Fully Connected Layers
     ↓
Output
```

AlexNet uses several convolutional layers followed by fully connected layers.

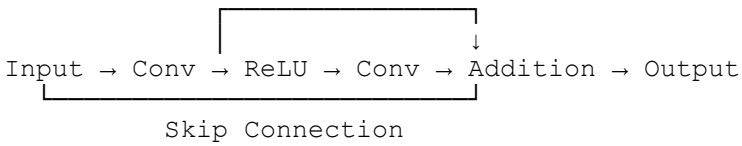
ResNet

ResNet, or **Residual Network**, uses many convolutional layers organized into **residual blocks**.

```
Input
  ↓
Convolution
  ↓
Residual Block
  ↓
Residual Block
  ↓
Residual Block
  ↓
...
  ↓
Global Average Pooling
```

```
  ↓
Fully Connected
  ↓
Output
```

The key innovation is the **skip connection**.



---

2. Network Depth

Architecture	Depth
AlexNet	Relatively shallow
ResNet	Can be much deeper

AlexNet contains around **8 learnable layers**, while ResNet is available in variants such as **ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152**.

Greater depth allows ResNet to learn more complex hierarchical representations.

```
AlexNet:
Edges → Textures → Shapes → Objects

ResNet:
Edges → Textures → Parts → Complex Structures → Objects
```

---

3. Layer Organization

AlexNet

AlexNet mainly follows a sequential architecture:

```
Conv → ReLU → Pool
        ↓
      Conv
        ↓
     ReLU → Pool
        ↓
    Fully Connected
```

ResNet

ResNet organizes layers into **residual blocks**.

The input can bypass one or more convolutional layers through a shortcut connection.

This allows the network to learn a residual mapping rather than an entire transformation.

---

4. Convolutional Filters

Both architectures use convolutional filters to automatically learn visual features.

AlexNet

Early filters learn:

- Edges
- Corners
- Simple textures

Deeper filters learn:

- Shapes
- Object parts
- High-level object features

ResNet

ResNet performs similar hierarchical feature learning but has many more layers available.

```
Early Layers
     ↓
Edges
     ↓
Textures
     ↓
Shapes
     ↓
Object Parts
     ↓
Complex Object Features
     ↓
Classification
```

Because of its depth, ResNet can learn more sophisticated representations.

---

5. Parameter Requirements

AlexNet contains a large number of parameters, with a substantial portion coming from its fully connected layers.

ResNet uses convolutional layers and typically replaces very large fully connected sections with **global average pooling**, which can greatly reduce parameter requirements relative to its depth.

For example:

Model	Approx. Parameters
AlexNet	~60 million
ResNet-18	~11.7 million
ResNet-50	~25.6 million
ResNet-101	~44.5 million
ResNet-152	~60.2 million

Therefore, **ResNet-18 can be considerably smaller than AlexNet while still providing a deeper architecture**.

---

6. Feature Extraction Capability

AlexNet

AlexNet is capable of learning useful hierarchical features, but its relatively shallow architecture limits how complex those representations can become.

ResNet

ResNet's deeper structure allows it to learn more detailed and hierarchical features.

For a complex image:

```
Pixels
  ↓
Edges
  ↓
Textures
  ↓
Shapes
  ↓
Object Parts
  ↓
Complex Object Representation
  ↓
Class
```

This makes ResNet particularly suitable for difficult image recognition tasks.

---

7. Training Difficulty

AlexNet

Because it is relatively shallow, AlexNet is comparatively easier to train.

ResNet

A very deep network would normally be difficult to train because of problems such as:

- Vanishing gradients
- Degradation of training accuracy
- Difficult gradient propagation

ResNet solves much of this difficulty using **skip connections**.

---

8. Vanishing-Gradient Problem

In very deep neural networks, gradients can become extremely small as they are propagated backward through many layers.

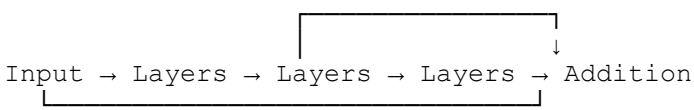
This is known as the **vanishing-gradient problem**.

AlexNet

Since AlexNet is relatively shallow, the problem is less severe.

ResNet

ResNet specifically addresses the difficulty of training very deep networks using shortcut connections.



The shortcut provides a more direct path for gradient propagation.

Therefore, ResNet can successfully train much deeper networks.

---

9. Computational Cost

AlexNet

Advantages:

- Relatively simple
- Lower depth
- Can have lower computational cost depending on implementation

However, its large fully connected layers contribute significantly to its parameter count.

ResNet

Deeper ResNet models require more computation.

For example:

```
ResNet-18  → Lower cost
ResNet-50  → Moderate cost
ResNet-101 → Higher cost
ResNet-152 → Very high cost
```

Therefore, the specific ResNet variant should be selected according to available hardware and latency requirements.

---

10. Classification Performance

AlexNet achieved a major improvement in image classification when introduced and remains useful as a historical baseline.

However, for modern complex image recognition, **ResNet generally provides stronger feature representations and better classification performance** because of its depth and residual architecture.

ResNet has also been widely used as a backbone for:

- Image classification
- Object detection
- Image segmentation
- Face recognition
- Medical image analysis

---

11. Overall Comparison

Feature	AlexNet	ResNet
Network depth	Relatively shallow	Deep/very deep
Organization	Sequential CNN	Residual blocks

Feature	AlexNet	ResNet
Skip connections	No	Yes
Feature extraction	Good	Excellent for complex tasks
Parameters	~60M	Depends on variant
Training	Relatively easier	Easier for deep networks due to skip connections
Vanishing gradients	Less severe due to shallower depth	Strongly addressed through residual connections
Computational cost	Moderate	Depends on depth; deeper variants are expensive
Classification performance	Good baseline	Generally stronger
Modern complex recognition	Less preferred	Highly suitable

---

12. Recommendation

For a **complex image recognition application, ResNet is the more suitable architecture**.

Reasons:

1. Better feature learning
Its deeper architecture can learn complex hierarchical representations.

2. Easier deep-network training
Residual connections help gradients flow through the network.

3. Reduced degradation problem
Adding more layers does not necessarily cause the same performance degradation seen in plain deep networks.

4. Parameter efficiency
Some ResNet variants, such as ResNet-18, achieve substantial depth with relatively few parameters.

5. Strong classification performance
ResNet generally provides better performance on challenging image-recognition problems.

6. Flexible architecture
Different variants allow a trade-off between accuracy and computational cost.

Final Recommendation

```
Complex Image Dataset
```

```
                ↓
          ResNet CNN
                ↓
Residual Feature Learning
                ↓
Deep Feature Extraction
                ↓
 Global Average Pooling
                ↓
     Classification
                ↓
      Predicted Class
```

ResNet should be preferred over AlexNet for complex image recognition because it provides **deeper feature extraction, better gradient flow, stronger classification performance, and better parameter efficiency in suitable variants**. If real-time or resource constraints are important, **ResNet-18 or ResNet-34** would provide a good balance between computational cost and recognition accuracy.

Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization and a CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Q5. Regularized CNN vs Non-Regularized CNN

For plant disease classification, the main problem is **overfitting**. The initial CNN achieves very high training accuracy but performs poorly on unseen leaf images, which means the model has learned the training images too closely instead of learning general disease patterns.

A **regularized CNN using dropout, data augmentation, and batch normalization** is therefore more suitable for reliable real-world classification.

---

1. Non-Regularized CNN

A non-regularized CNN may have the following structure:

```
Leaf Image
    ↓
Convolution
    ↓
```

```
ReLU
    ↓
Pooling
    ↓
Convolution
    ↓
ReLU
    ↓
Pooling
    ↓
Fully Connected
    ↓
Disease Class
```

If the model is too complex relative to the training data, it may **memorize specific leaf images**.

For example, it may learn:

- Exact leaf backgrounds
- Specific lighting conditions
- Camera characteristics
- Individual leaf shapes

instead of general disease characteristics.

Result

```
Training Accuracy → Very High
Validation Accuracy → Low
Testing Accuracy → Low
```

This is a classic indication of **overfitting**.

---

2. Regularized CNN

A regularized CNN introduces techniques that encourage the model to learn generalizable patterns.

```
            Leaf Image
                ↓
         Data Augmentation
                ↓
         Convolution Layer
                ↓
      Batch Normalization
                ↓
              ReLU
                ↓
         Pooling
                ↓
         Convolution Layer
                ↓
```

```
Batch Normalization
        ↓
      ReLU
        ↓
    Pooling
        ↓
    Dropout
        ↓
 Classification Layer
        ↓
   Plant Disease
```

The three techniques serve different purposes.

---

3. Dropout

Dropout randomly deactivates a fraction of neurons during training.

For example:

```
Without Dropout:
```

• • • • • • •

```
With Dropout:
```

• × • • × • •

The model therefore cannot rely too heavily on a small set of neurons.

Benefits

- Reduces overfitting
- Encourages independent feature learning
- Improves generalization

For example, dropout can be applied before the final classification layer.

Limitation

Too much dropout can cause **underfitting** and slow down learning.

---

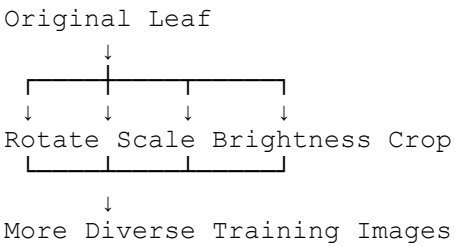
4. Data Augmentation

Data augmentation is especially important for plant disease classification because real-world leaf images can vary greatly.

Training images can be modified using realistic transformations such as:

- Rotation
- Horizontal/vertical flipping when appropriate
- Scaling
- Cropping
- Translation
- Brightness changes
- Contrast changes
- Small amounts of noise

Example:



Benefits

The model learns that a disease is not defined by a particular:

- Orientation
- Position
- Lighting condition
- Scale

This improves **generalization to unseen images**.

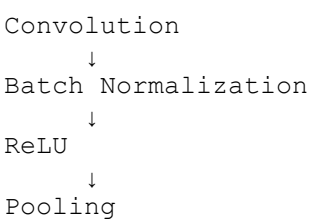
For agricultural applications, augmentation should remain realistic. For example, transformations that produce biologically unrealistic leaf images should be avoided.

---

5. Batch Normalization

Batch normalization normalizes the activations produced by a layer.

A typical CNN block can be:



Benefits

- Stabilizes training
- Improves gradient flow
- Allows faster convergence
- Makes training less sensitive to initialization
- Provides some regularization effect

Batch normalization is primarily a **training stabilization technique**, although it can also contribute to regularization.

---

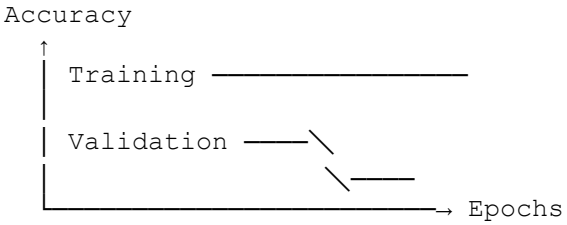
6. Comparison: Regularized vs Non-Regularized CNN

Factor	CNN Without Regularization	CNN with Regularization
Overfitting	High risk	Lower risk
Training accuracy	Often very high	High, but usually more realistic
Validation accuracy	Often low	Usually improved
Generalization	Poor	Better
Training stability	May be less stable	More stable with batch normalization
Training time	Usually lower	May increase slightly
Computational requirements	Lower	Slightly higher
Robustness	Lower	Higher
Real-world performance	Poorer	Better
Model reliability	Lower	Higher

---

7. Effect on Training and Testing

Without Regularization



The training accuracy continues increasing while validation performance remains poor or begins to decrease.

With Regularization

Training and validation performance are generally closer, indicating better generalization.

---

8. Computational Requirements

Regularization introduces some additional computation.

Data Augmentation

Images must be transformed during training, increasing preprocessing requirements.

However, it usually does **not significantly increase inference cost**, because augmentation is normally used only during training.

Dropout

Dropout adds little computational overhead during training and is normally disabled during inference.

Batch Normalization

Batch normalization adds normalization operations during training and inference, but modern CNN implementations handle these operations efficiently.

Therefore, the additional cost is generally justified by improved model reliability.

---

9. Which Techniques Should Be Applied?

For this agricultural application, **all three techniques can be used together**, but each addresses a different issue.

1. Data Augmentation — Highly Recommended

This should be a priority because real-world leaf images can have variations in:

- Lighting
- Orientation
- Scale
- Position

- Background

It directly increases the diversity of training examples.

2. Batch Normalization — Recommended

Use batch normalization after convolution and before/around activation in appropriate CNN blocks.

It improves training stability and can provide some regularization.

3. Dropout — Recommended Carefully

Use moderate dropout, particularly near the classification head.

For example:

```
Feature Extraction
       ↓
Global Average Pooling
       ↓
Dropout
       ↓
Classification
```

The exact dropout rate should be selected using validation performance rather than choosing an arbitrary value.

---

10. Recommended Architecture

A practical architecture would be:

```
                Leaf Image
                     ↓
            Data Augmentation
                     ↓
             Convolution
                     ↓
        Batch Normalization
                     ↓
                  ReLU
                     ↓
                Pooling
                     ↓
             Convolution
                     ↓
        Batch Normalization
                     ↓
                  ReLU
                     ↓
                Pooling
                     ↓
             Deeper CNN Layers
```

```
                ↓
   Global Average Pooling
                ↓
          Dropout
                ↓
      Classification
                ↓
Plant Disease Category
```

For a larger application, a **pretrained CNN such as ResNet** can also be fine-tuned with these regularization techniques.

---

11. Why Regularization Improves Real-World Disease Detection

A plant disease classifier must work on images captured under conditions different from the training dataset.

For example:

```
Training:
Clear image + controlled lighting
          ↓
       CNN Model
          ↓
Testing:
Outdoor image + shadows + different camera
          ↓
    Reliable Prediction
```

Regularization helps the CNN focus on **actual disease characteristics** such as:

- Spots
- Discoloration
- Lesions
- Texture changes
- Leaf damage

rather than memorizing the background, lighting, or individual training leaves.

Final Recommendation

The **regularized CNN is clearly preferable** to the non-regularized CNN for reliable plant disease classification.

The recommended approach is:

Data Augmentation + Batch Normalization + Moderate Dropout

because:

- **Data augmentation** improves data diversity and generalization.
- **Batch normalization** stabilizes and speeds up training while providing some regularization.
- **Dropout** reduces dependence on individual neurons and helps prevent overfitting.

Therefore, the final model should aim for a **small gap between training and validation performance**, rather than simply maximizing training accuracy. This produces a CNN that is more robust to real-world variations and more reliable for **unseen plant leaf images**.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Selection of Studies/Parameters	**20%**	**Selects highly relevant and diverse studies with well-defined comparison parameters**	**Selects relevant studies with minor gaps in parameter definition**	**Limited or partially relevant selection**	**Poor or inappropriate selection of studies**
Comparison & Differentiation	**25%**	**Clearly compares multiple aspects (methods, results, limitations) with strong differentiation**	**Good comparison with minor gaps**	**Basic comparison with limited depth**	**No clear comparison; mostly descriptive**
Critical Evaluation	**25%**	**Critically evaluates strengths, weaknesses, and implications with strong justification**	**Provides evaluation with some justification**	**Limited evaluation; lacks depth**	**No critical evaluation provided**

Synthesis & Insight Generation	**20%**	**Generates meaningful insights and conclusions from comparisons**	**Some insights derived from comparison**	**Limited or unclear insights**	**No insights generated**
Clarity	**10%**	**Information is clearly presented (tables/figures) with logical structure**	**Generally clear with minor issues**	**Presentation lacks clarity or structure**	**Poor presentation and difficult to understand**